Project Documentation

ArmDealer

Recognitions

Developer:
Conel, Kelsen Gile S.

Course:
Bachelor of science in Computer Science

Subject:
ITEC 103
Intermediate Programming

Submitted to:
Francisco Kaleb Marquez

Institution:
Laguna State Polytechnic University
College of Computer Studies

Date Submitted
May, 30 2026

Table of Contents

Title **1**
Recognitions **2**
Table of Contents **3**
Introduction **4**
Purpose 4
Objective 5
Project Status **5**
Features **6**
System Requirements **23**
Installation and Setup **24**
Development Accounts **25**
Known Limitations **26**
Recommendations **26**
Security Considerations **28**
Performance Considerations **28**
Scalability Considerations **29**
Future Enhancements **29**
Brands Catalogue Reference **31**
Categories and Products Reference **32**
Database Maintenance **33**
Contribution Notes **34**
Definition of Terms **34**
Bibliography **38**

Introduction

ArmsDealer.com is a Flask-based e-commerce web application built for the sale, browsing, and management of arms, tactical equipment, and related professional services. It is designed as a full-stack platform that covers the entire retail lifecycle from product discovery to order fulfillment, with a built-in administration system for complete backend control.

The platform is built with a military-aesthetic interface featuring a dark color scheme, customizable accent colors, scanline overlays, background imagery, and a monospace typographic style that reinforces the tactical branding of the platform. All of these visual elements are customizable by individual users through the appearance settings system, and are rendered server-side on every request to ensure visual consistency without any flash of unstyled content.

Purpose

The purpose of ArmsDealer.com is to provide a dedicated, self-contained e-commerce environment tailored to the arms and security industry — a sector with unique requirements that general-purpose retail platforms do not address. The platform is built to handle both civilian-authorized products and restricted listings, with access control that automatically adjusts what each visitor can see based on their authentication status. It supports the full transactional workflow including cart management, multi-currency checkout, order tracking, and post-delivery item ratings, while also giving administrators a centralized dashboard to manage inventory, orders, customers, and support inquiries without relying on any third-party backend service.

Beyond commerce, the platform is designed to serve as a trusted storefront for professional buyers — security firms, private contractors, law enforcement procurement officers, and individual operators — who require a secure and organized interface for sourcing weapons, equipment, ammunition, tactical gear, and specialized services such as training, maintenance, transport, and consulting. The authorization tier system ensures that sensitive or restricted product categories remain visible only to verified, logged-in users, while the public-facing catalogue presents a curated selection to unauthenticated visitors.

Objective

The primary objective of ArmsDealer.com is to deliver a fully functional, production-ready e-commerce platform that serves as the complete digital storefront and backend management system for an arms and security products business. Specific objectives of the project include the following.

To build a secure and reliable authentication system that supports email-verified registration, OTP-based password recovery, two-factor authentication, active session management, and a full login history audit trail, ensuring that all user accounts are protected against unauthorized access.

To implement a comprehensive product and service catalogue with multi-language translation support, multi-currency pricing, hierarchical categorization across weapons and equipment types, brand association, and a tiered authorization model that segregates public and restricted inventory.

To provide a complete commerce workflow covering cart management, checkout with multiple payment methods, order lifecycle tracking through all fulfillment stages, automatic inventory deduction, and email notifications at every significant order event.

To give administrators a powerful, all-in-one dashboard that eliminates the need for direct database access during day-to-day operations, covering inventory CRUD, order status management, user account administration, support inquiry handling, and real-time sales analytics.

To allow users to personalize their experience through a deeply customizable appearance system, notification preferences, and account profile management, increasing engagement and usability across different user types and preferences.
To lay a clean and extensible technical foundation that future developers can build on, with modular blueprint-based routing, a well-defined database schema with proper indexing and trigger logic, a centralized email service, and clear separation between authentication, public routes, cart logic, and API endpoints.

Project Status

The project is in an active development state. The following reflects the current completion status of major areas:

Complete and functional:

- User authentication (registration with OTP, login, logout, forgot password, change password)
- Product catalogue with category and brand browsing
- Specific product detail pages with multi-image galleries and related products
- Shopping cart (add, remove, quantity management)
- Checkout with cash-on-delivery and wallet payment methods
- Order management for both customers and administrators
- Full admin dashboard with all CRUD operations and analytics
- User account settings (all seven settings tabs)
- Appearance customization system
- Multi-currency support
- Global search panel
- Contact and support inquiry system
- Email notification system
- Two-factor authentication
- Active session management
- Login history tracking
- Notification preferences
- Account data export
- Account deactivation and deletion
- Legal, about, and contacts static pages

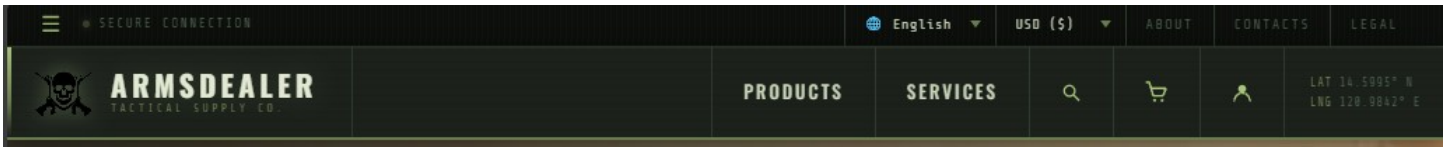
In progress or not yet started:

- Promotions system (bundles, seasonal sales, vouchers, exclusive drops)
- Services browsing page with authorized/restricted filtering
- Specific service detail pages
- Tools section in the navigation
- Updates and news section
- Homepage best sellers section

- Homepage news and updates section
- Trusted brands carousel on homepage
- Product ranking and sorting controls on the products page
- Appearance settings panel (the settings panel UI is partially complete in code)
- Notification settings panel (backend is complete; frontend panel integration may be partial)

Features

Navigation Bar



1.1 NavBar (Primary Navigation)

The main horizontal navigation bar fixed at the top of every page rendered via the shared base.html template.

- Logo: Displays the ArmsDealer branding/logo image. Clicking it returns theuser to the homepage.

- Page Button: Products A navigation link that routes the user to the Products listing page (/products). Applies an active highlight class when the user is on the Products page.

- Page Button: Services A navigation link that routes the user to the Services page (/services). Applies an active highlight class when the user is on the Services page.

- Search A button that opens the Search Side Panel overlay, allowing users to search across all products and services.

- Cart A button that links to the Orders page (/orders), showing the user's current cart. Displays a live item count badge reflecting the number of items currently in the cart.

- Account Panel A button that opens the Account Side Panel overlay, giving the user quick access to account options, login/logout, and profile details.

1.2 TopBar (Secondary Navigation)

A slim bar rendered above the main NavBar, containing utility and secondary navigation controls.

- Settings Panel (Button) A hamburger/menu button (☰) on the left side of the TopBar that opens the Settings Side Panel overlay.

- Secure Connection Indicator A small status dot and label indicating the connection is secure, displayed next to the settings button.

- Language Selector

A dropdown (select element) allowing the user to switch the interface language. Supported languages: English, Filipino, Japanese, Spanish, and Mandarin. The entire site re-renders its translatable text strings based on the chosen language via a translations.js system and data-translate attributes.

- Currency Preference

A dropdown (select element) allowing the user to switch the active display currency. Supported currencies: PHP (₱), USD ($), EUR (€), JPY (¥), CNY (¥). All product prices across the site are recalculated and displayed in the selected currency using stored exchange rates. The selected currency is persisted in a cookie.

- Page Button: About

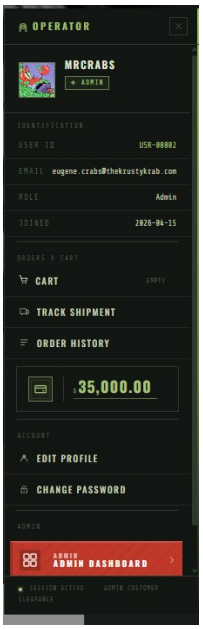
  A navigation link in the TopBar that routes the user to the About page (/about).

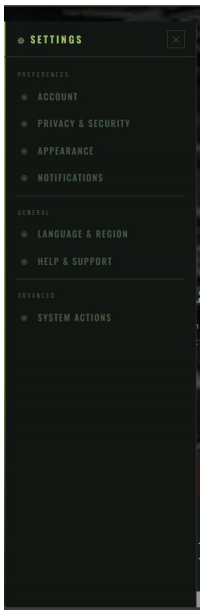
- Page Button: Contacts

  A navigation link in the TopBar that routes the user to the Contacts page (/contacts).

- Page Button: Legal

  A navigation link in the TopBar that routes the user to the Legal page (/legal).





1.3 Side Panels

Slide-in overlay panels accessible from the NavBar and TopBar controls.

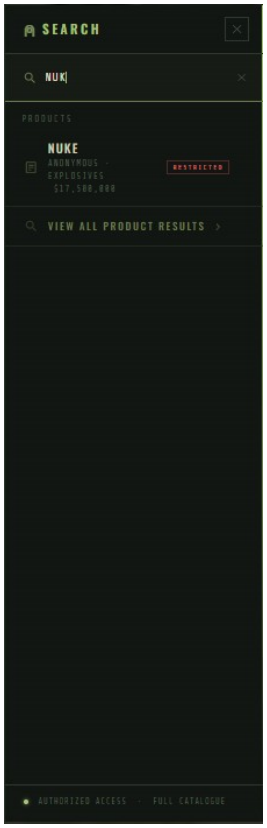
All three panels are included in every page via the base template.

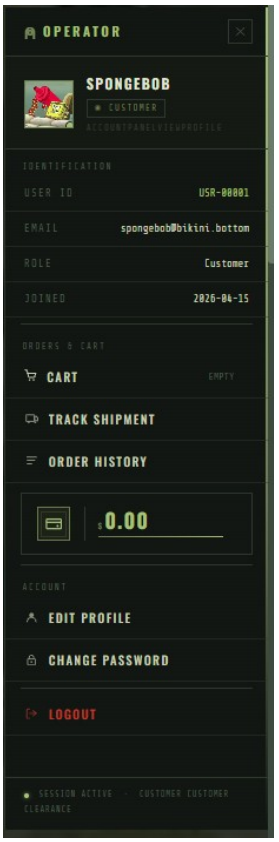
- Settings Side Panel

  A slide-in panel opened via the TopBar settings button. Contains all user-configurable appearance and system settings (see Section 8).

- Account Side Panel

  A slide-in panel opened via the NavBar account button. Displays the logged-in user's profile picture, username, and role. Provides quick links to the Account page, Orders page, Settings page, Admin Dashboard (if admin), and a Logout button. If the user is not logged in, it shows login and register links instead.

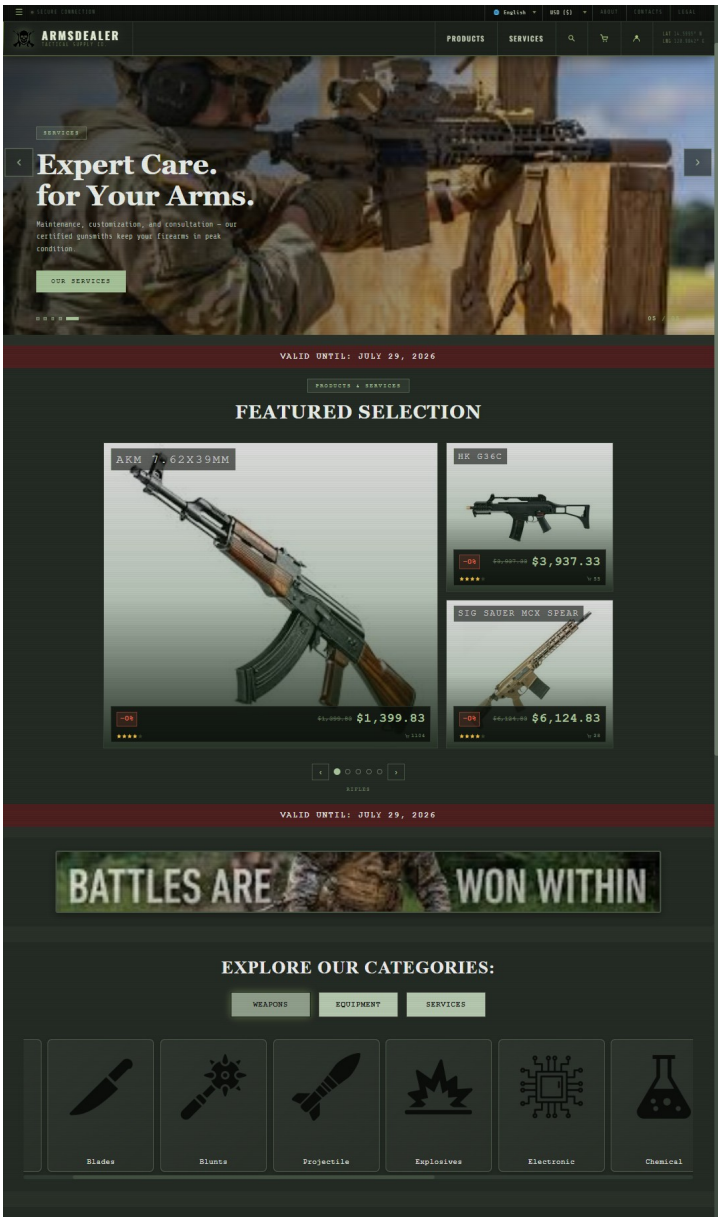


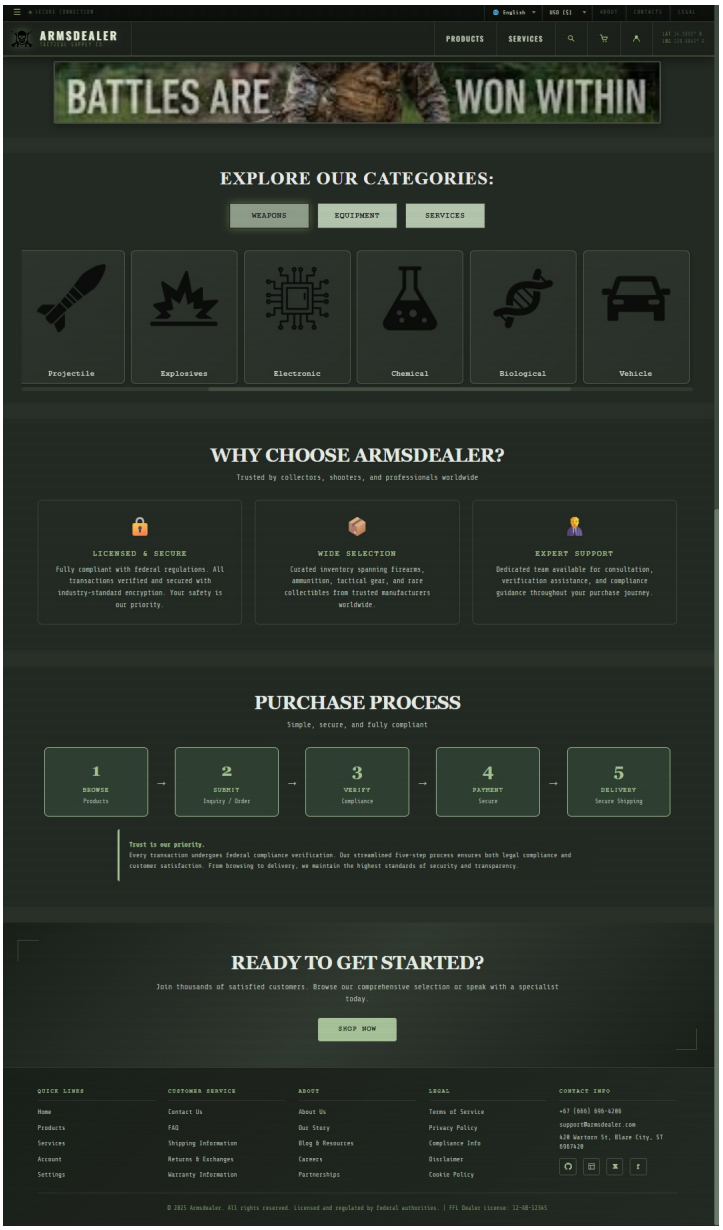


- Search Side Panel

  A slide-in panel opened via the NavBar search button. Contains a search input field. As the user types, it queries the backend API and displays matching products and services in real time, each with an image thumbnail, name, price, and a link to the specific item's page.

Home Page





The landing page of the site, designed to orient and engage new and returning visitors.

- Hero Image Carousel

A full-width, auto-advancing image carousel (5 slides) at the top of the homepage. Each slide features a background image, a tag label, a headline title, a subtitle, and a call-to-action button linking to either Products or Services. The carousel supports manual navigation via previous/next arrow buttons, clickable dot indicators, and a current slide counter (e.g., "01 / 05").

- Featured Selection (Feature Carousel)

A curated product showcase section that groups handpicked products into category tabs (e.g., Rifles, Pistols, etc.). Each tab displays a collage of product cards with images, product names, star ratings, sales counts, original prices, discounts, and discounted prices. Clicking a card opens a quick-view popup modal with product details and an Add to Cart button. Prices are displayed in the user's selected currency.

- Ad Space Section

A dedicated promotional banner section on the homepage. Displays a full-width advertisement image with accompanying marketing copy and a call-to-action button linking to the Products page.

- Categories Section

A visual grid of product/service categories, each represented by an icon and a label. Clicking a category navigates the user to the Products page filtered by that category.

- Onboarding Section

An introductory section aimed at new users, briefly explaining the platform's purpose and key value propositions to help visitors understand what ArmsDealer.com offers.

- Purchase Process Section

A step-by-step visual guide explaining how to complete a purchase on the site, walking the user through the buying flow from browsing to checkout.

- Redirect Section

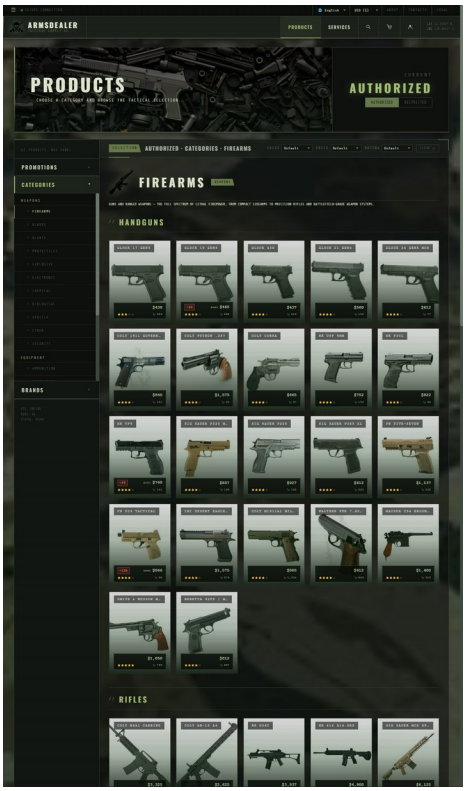
A section containing prominent call-to-action buttons or banners that redirect users to key areas of the site, such as the Products page and the Services page.

- Footer

The page footer included at the bottom of the homepage (and other pages).Contains site branding, navigation links, legal disclaimers, and other supplementary information.

Products Page

The main product browsing and discovery page.



- Hero Image

A banner image at the top of the Products page establishing the visual theme of the section.

- Legality Filter

A toggle or filter control that allows users to filter the product listing between authorized/regulated items and restricted items, helping users find products appropriate for their jurisdiction or clearance level.

- Products Navigation Bar

A secondary navigation bar specific to the Products page, containing:

- Promotions

A filter or tab that surfaces currently discounted or promotional products.

- Categories

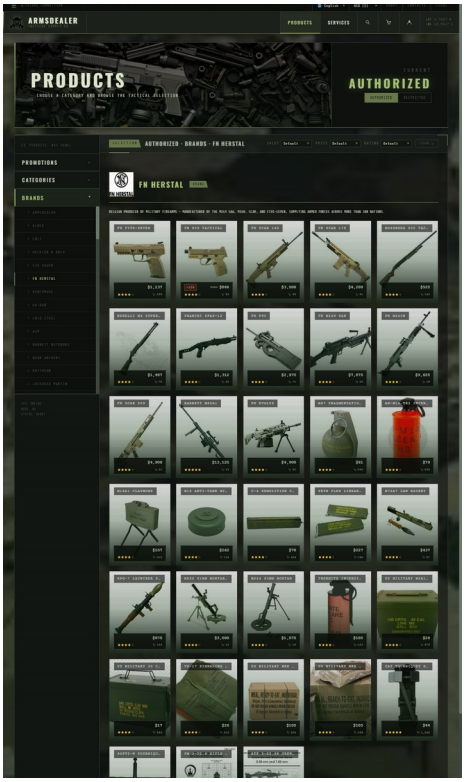
A dropdown or expandable menu listing all product categories and subcategories, allowing users to narrow results to a specific type of item (e.g., Firearms, Ammunition, Protective Gear, Blades, etc.).

- Brands

  A dropdown or filter listing all available brands, allowing users to filter the product listing to a specific manufacturer or label.

- Content Panel

  The main product listing area, divided into:



  - Top Bar

    A control bar at the top of the product listing containing:

    - Content Identifier

      A label or breadcrumb indicating which category, brand, or filter is currently active, so the user knows what they are browsing.

    - Content Filters

      Sorting and filtering controls (e.g., sort by price, rating, sales count; filter by availability or legality status) that refine the product listing in real time.

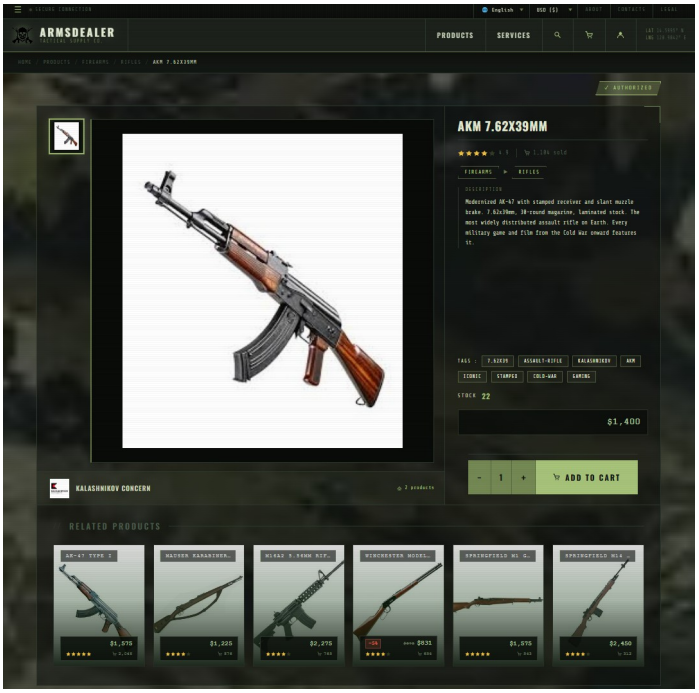
  - Content Section

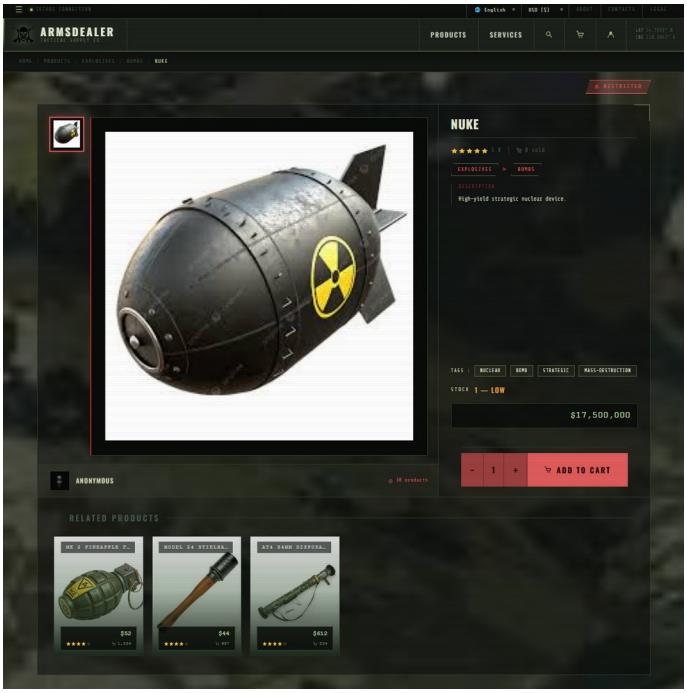
    The scrollable grid of product cards. Each card displays a product image, name, brand, price (in the selected currency), discount percentage, discounted price, star rating, and a quick-add or view button. Clicking a card navigates to the specific product's detail page.

- Specific Product Page (/product/<slug>)

  A dedicated detail page for an individual product, including a multi-image gallery, full product description, category and brand information, brand logo, price and discount, star rating, sales count, legality status, related products section, and Add to Cart functionality.

Specific Product Page





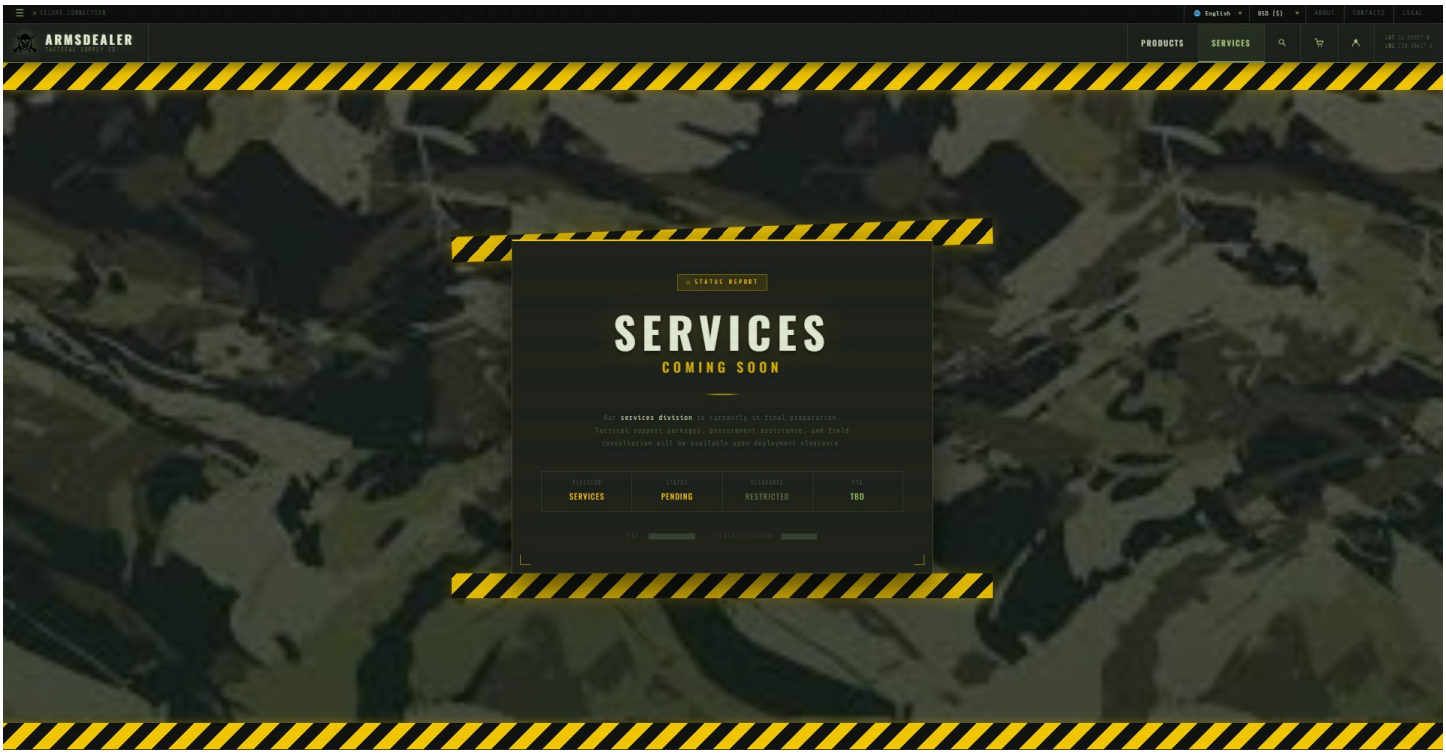
The specific product page is the dedicated detail view for an individual product, accessible via its unique URL slug (e.g., /product/glock-19-gen5). It renders the specificproduct.html template and is served by the product_detail route in main_routes.py.

The page is divided into two main sections. The upper section contains a two-column layout: a left-side image gallery with a scrollable thumbnail strip and a large active image viewer, paired with a brand panel below it that links back to the brand's full catalog; and a right-side information panel displaying the product name, star rating, sales count, category/subcategory breadcrumb pills, an inline description, tags, current stock status, pricing (with discount badge and original price strikethrough if applicable), and a quantity selector with an Add to Cart button (or a disabled Sold Out state if stock is zero). The page also displays an access badge — ✓ AUTHORIZED or ⚠ RESTRICTED — depending on whether the current user's account tier is cleared to view the product. Restricted products render the page in a restricted-mode class that visually limits certain interactions.

Below the main grid, a Related Products section displays up to six cards pulled from the same brand/subcategory combination, each linking to their own product page.

Pricing throughout the page is dynamically converted from a base USD price to the user's selected currency using a rate stored in the currency context object. Product data — including translations, brand info, category hierarchy, and additional images — is fetched from the SQLite database via a single joined query at page load.

Service Page



- Coming Soon

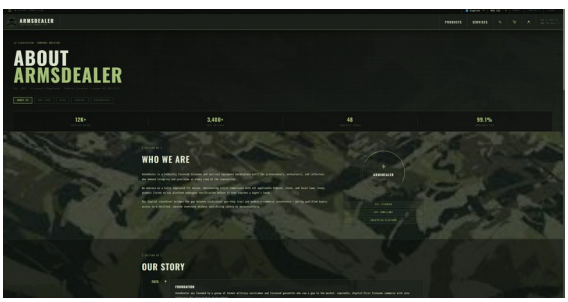
The Services page currently displays a "Coming Soon" placeholder, indicating the section is planned but not yet fully implemented. The page uses its own stylesheet (servicesstyles.css) and is registered in the routing system.

- Specific Service Page (/service/<slug>)

A dedicated detail page template (specificservice.html) exists for individual services, structured similarly to the specific product page.

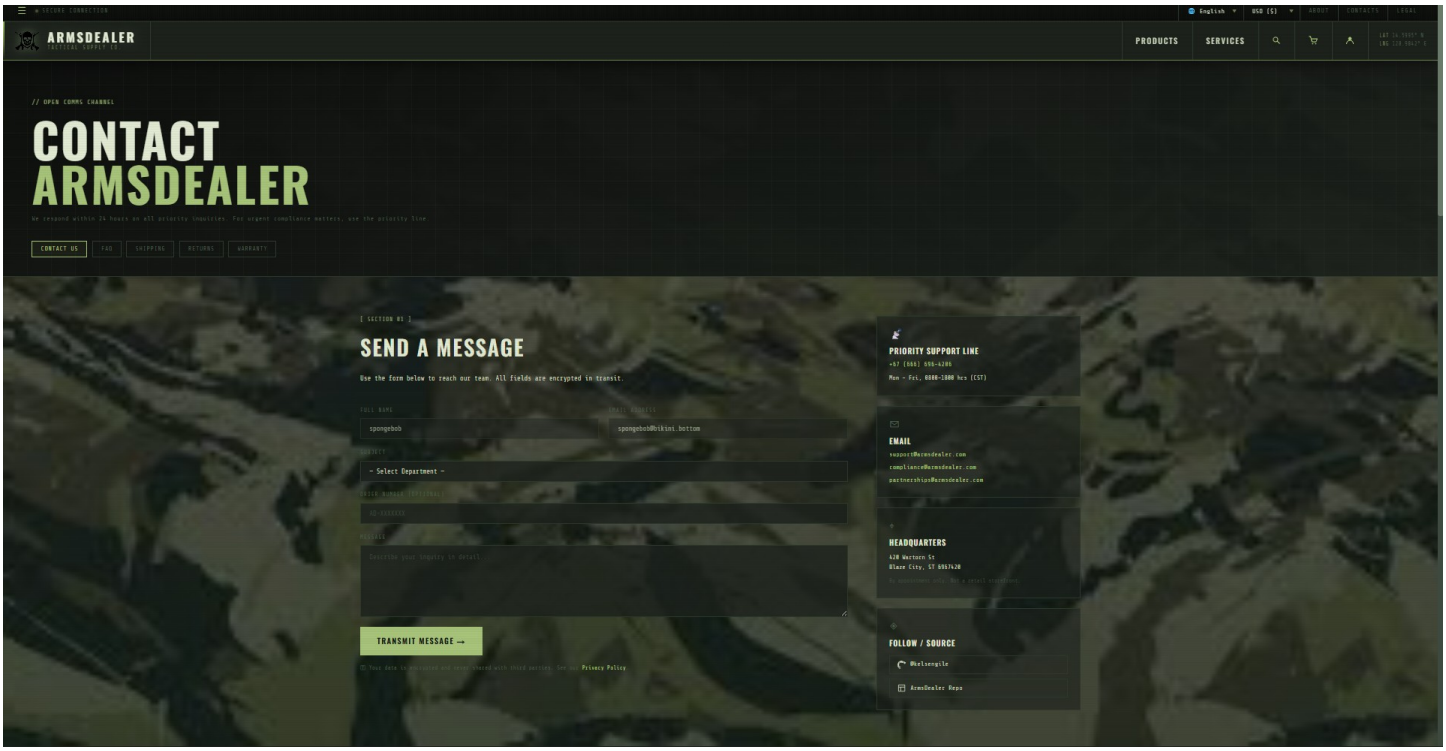
About Page

An informational page describing the platform, its mission, background, and the team or organization behind ArmsDealer.com. Rendered from about.html



with its own stylesheet.

Contacts Page



A page where users can submit inquiries to the site administrators and view the history of their past inquiries.

- Inquiry Submission Form

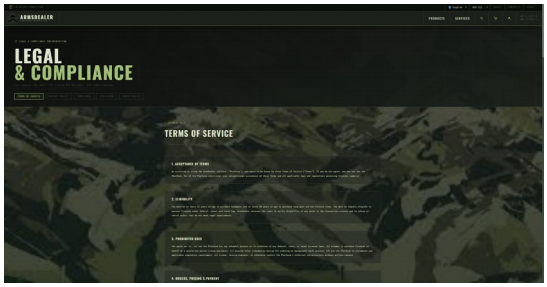
A form allowing visitors to send a message to the site team, including fields for name, email, subject, message, and optionally an order number for order-related inquiries.

- Past Inquiries (for logged-in users)

Logged-in users can see a list of their previously submitted inquiries, including the subject, status, admin replies (if any), and the date each inquiry was submitted. Up to 20 past inquiries are displayed.

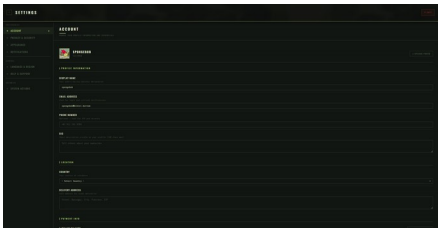
Legal Page

A page containing the site's legal documentation, including terms of service, disclaimers, usage policies, and any other regulatory or compliance content relevant to the sale of arms and related products/services.



Settings Page

A comprehensive user preferences and configuration page. For guest users, appearance settings are still accessible. Full account settings require login.

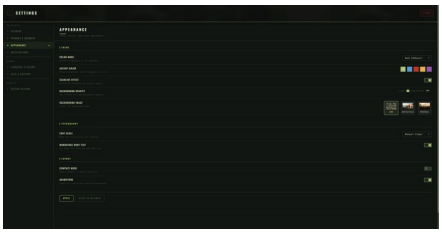


- Account (Settings Section)

  Allows the logged-in user to update their profile information including username, email address, profile picture, country, and delivery address.

- Privacy and Security

  Provides security-related settings including a login history log (showing the last 50 login attempts with timestamp, IP address, user agent, and success/failure status), and options related to account security.



- Appearance

  Controls the visual presentation of the entire site:

  - Color Mode: Dark (Default), Light, High Contrast
  - Accent Color: Olive, Steel, Red, Amber, Violet
  - Background Image: selectable from available background options
  - Background Opacity: adjustable slider (0–100%)
  - Font Scale: Small (10px), Default (12px), Large (14px)
  - Scanlines toggle: enables/disables a CRT scanline overlay effect
  - Monospace Body toggle: switches body text to a monospace font
  - Compact Mode toggle: reduces spacing throughout the UI
  - Animations toggle: enables/disables CSS transition animations

  All appearance settings are persisted in both localStorage (for instant apply on page load) and a server-side cookie (for server-rendered injection).

- Notifications

  Controls for managing email notification preferences, including which categories of notifications the user wishes to receive (e.g., order confirmations, status updates, login alerts) and quiet hours settings.

- Language and Region

  Settings for the user's preferred display language and currency, mirroring the TopBar controls but saved persistently to the user's preferences.

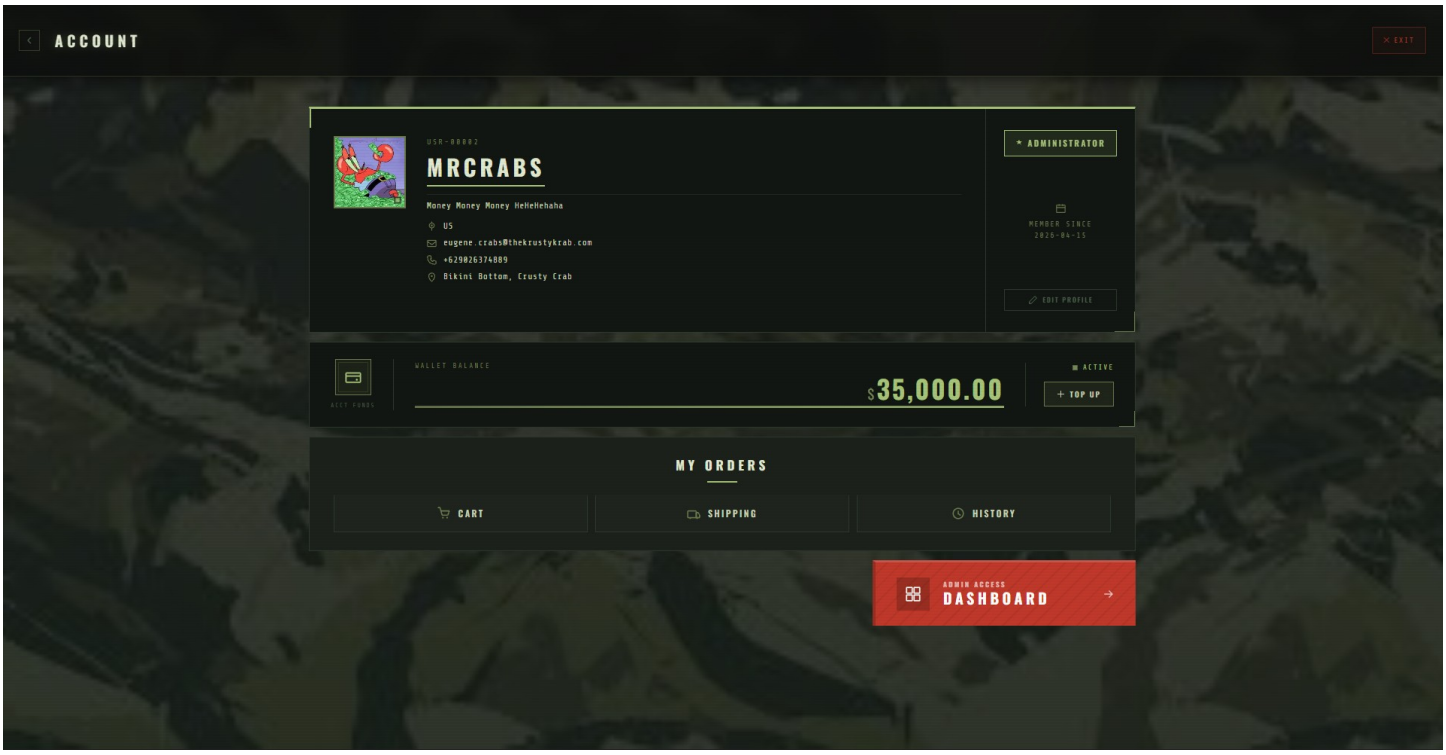
- Help & Support

  A section providing links or information to help the user get assistance, such as FAQs, support contact options, or documentation.

- System Actions

Dangerous or irreversible account-level actions, such as clearing stored data, resetting preferences to defaults, or initiating account deletion.

Account Page



A user-facing profile summary page. Requires login.

- Account Information

Displays the user's profile details: profile image, username, email, role (user or admin), country, and any other stored profile fields.

- Account Balance

Displays the user's current wallet balance, converted and shown in the active selected currency. The wallet balance is stored in PHP and converted at runtime using the stored exchange rate.

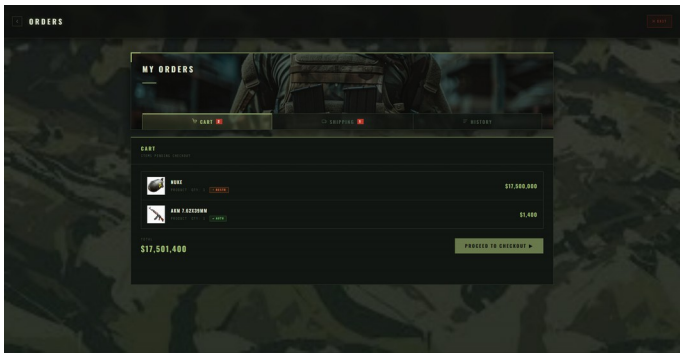
- Orders Page Redirect

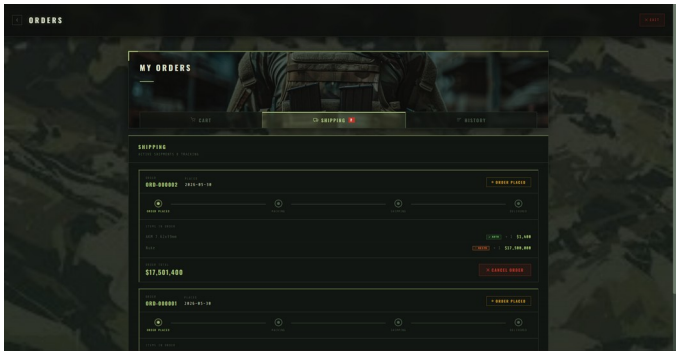
A prominent link or button redirecting the user to the Orders page (/orders) where they can view their cart, active orders, and order history.

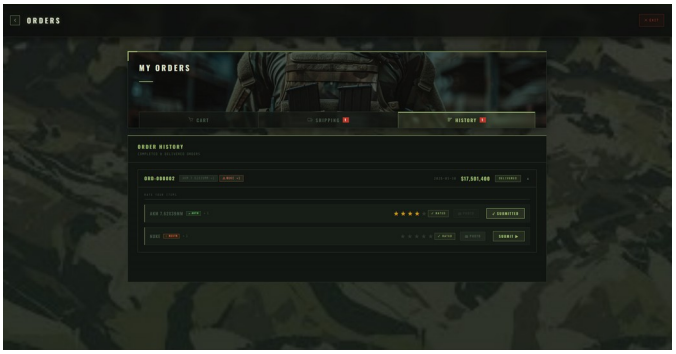
- Admin Dashboard Redirect

For users with the admin role, a link or button is displayed redirecting them to the Admin Dashboard (/dashboard).

Orders Page







A combined page for managing all order-related activities. Requires login.

- Cart

Displays all items currently in the user's cart, each showing the item image, name, legality status, quantity, and price in the selected currency. The cart total is calculated and displayed. Items in the cart can be removed or adjusted. A "Proceed to Checkout" action is available.

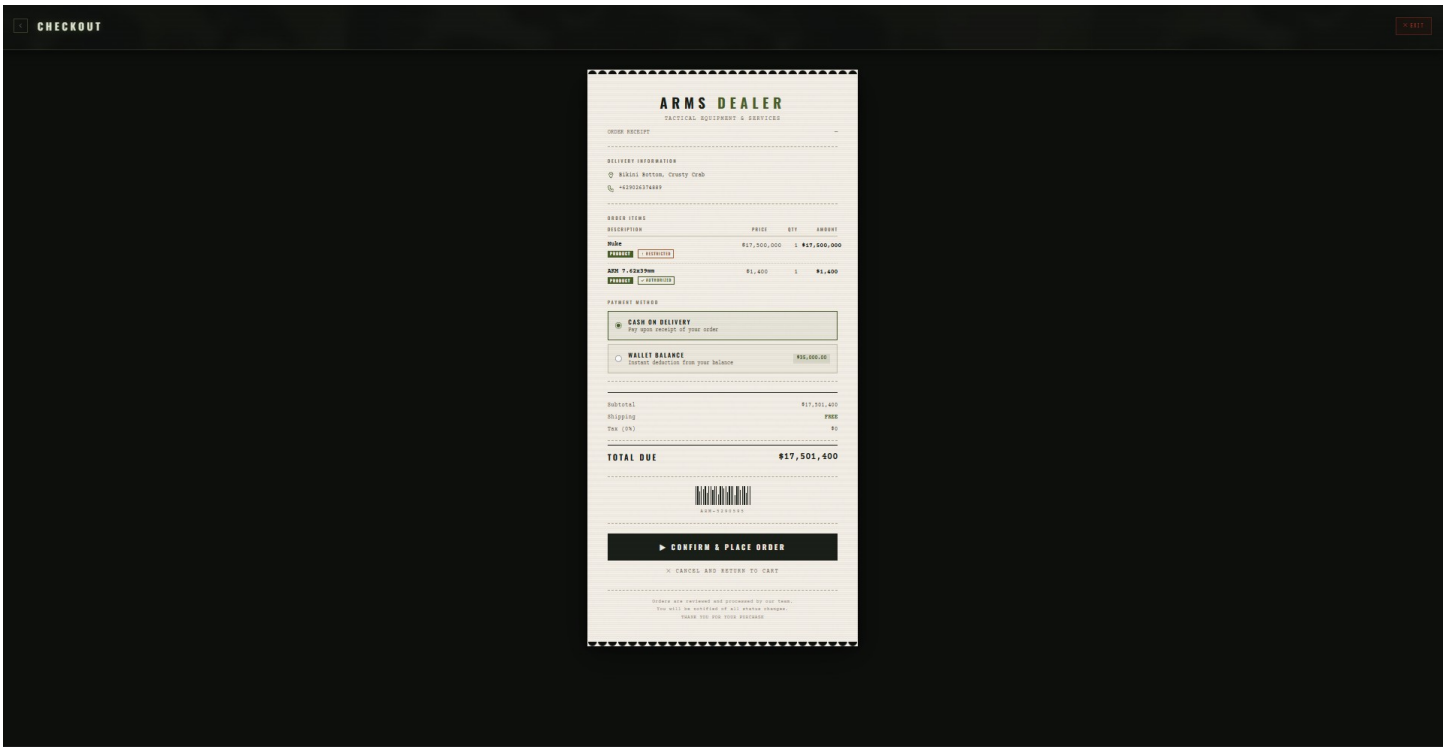
- Active Orders (Shipping)

Displays all current orders that have not yet been delivered, showing order ID, status (e.g., pending, processing, shipped), creation date, item list with quantities and prices, and the order total in the selected currency.

- History (Delivered Orders)

Displays all past delivered orders, including order details and a list of purchased items. For each delivered item, the user can submit a star rating (1–5 stars). Previously submitted ratings are pre-loaded and displayed. This section serves as the full purchase history for the user.

Checkout Page



The checkout process where the user reviews and confirms their purchase.

- Delivery Information

Displays the user's saved delivery address from their account settings. If no address is set, a warning is shown prompting the user to add one in Settings before placing the order.

- Order Items Summary

A receipt-style listing of all items being purchased, with item names, quantities, and unit prices.

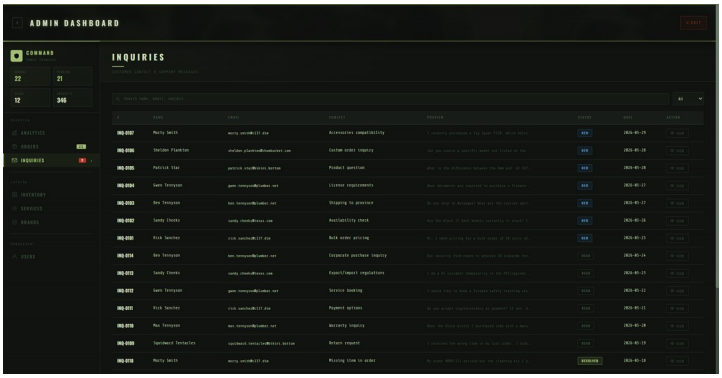
- Payment Method

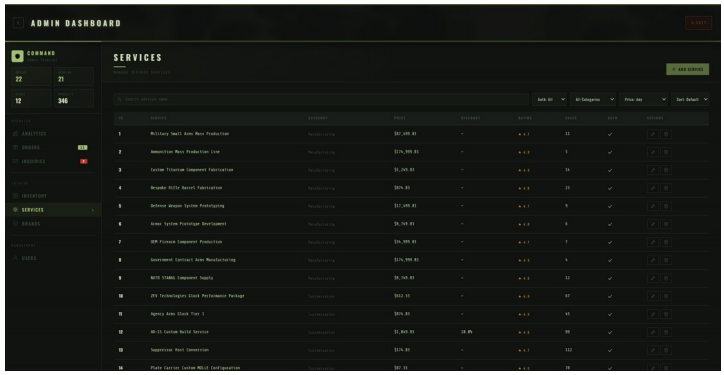
Displays the available payment options and allows the user to select their preferred payment method before finalizing the order.

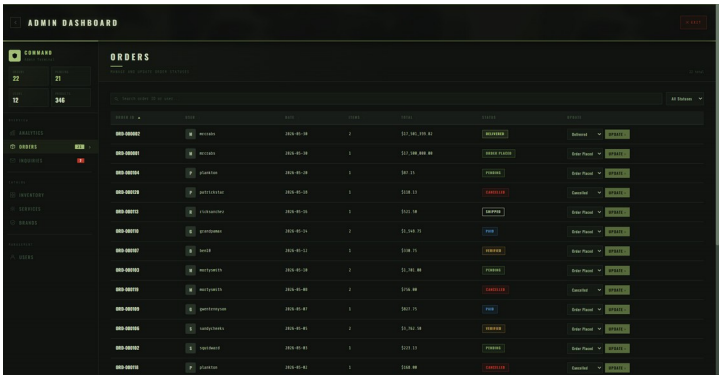
- Order Placement

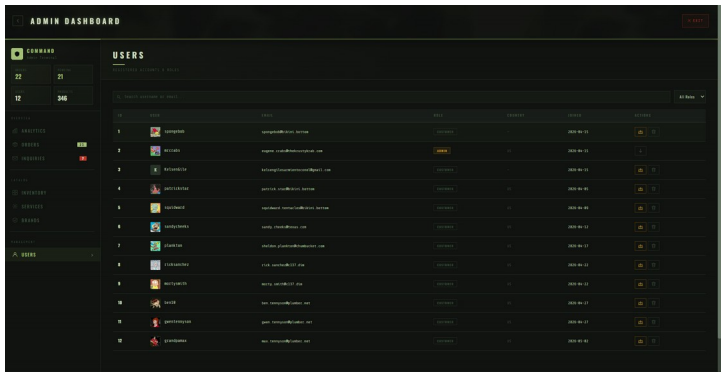
Confirms and submits the order, deducting from the wallet balance if applicable, creating the order record in the database, and triggering the order confirmation email notification.

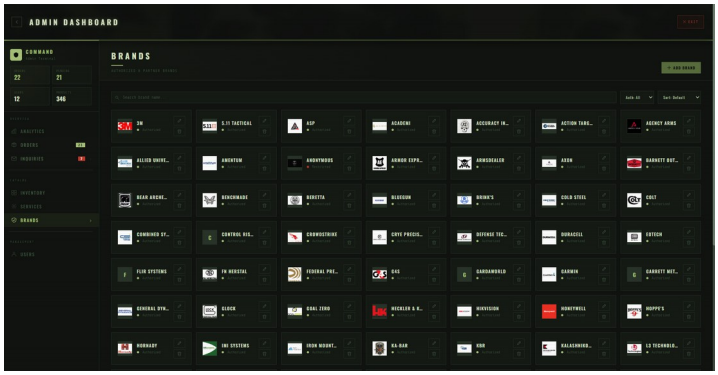
Admin DashBoard











A restricted page accessible only to users with the admin role. Provides full site management capabilities.

- Analytics

A real-time statistics panel displaying key metrics: total users, total orders, total products, total services, pending orders, delivered orders, and total revenue (sum of all delivered order totals). Also includes:

- A revenue-by-day bar or line chart for the last 7 days
- Top 6 products by sales count
- Top 6 products by revenue
- Category breakdown (product count per category, top 8)
- Low stock alerts (products with 5 or fewer units remaining)
- Recent users (last 5 registered accounts)

- Orders Management

A full table of all orders across all users, showing order ID, username, item count, status, total, creation date, and last update date. Admins can update order statuses (e.g., from pending to processing to shipped to delivered), which triggers an email status update notification to the customer.

- Inquiries

A list of all customer inquiries submitted via the Contacts page, with the ability for admins to read and reply to each inquiry. Reply content is stored and shown back to the customer on the Contacts page.

- Inventory (Products Management)

A full list of all products in the database with their name, category, brand, price, discount, stock count, rating, sales count, and legality status (is_authorized flag). Admins can add new products, edit existing products (including uploading up to 5 product images), and manage stock.

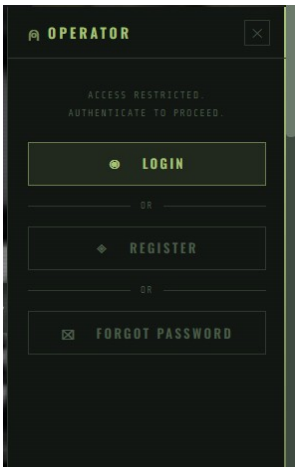
- Services Management A full list of all services, similar to the inventory panel, with the ability to add, edit, and manage service listings including legality status.

- Brands Management A list of all brands with their name, logo, slug, and authorization status. Admins can add new brands and edit or update existing ones, including uploading brand logo images.

- Users Management

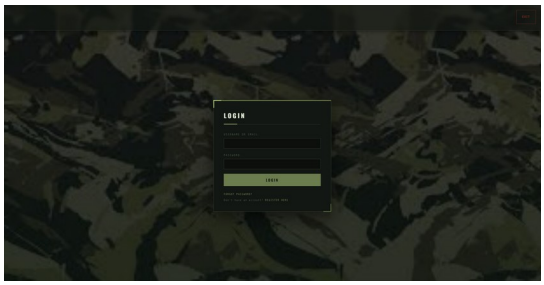
A list of all registered user accounts showing user ID, username, email, role, profile image, country, and registration date. Admins can view and manage user accounts.

Authentication



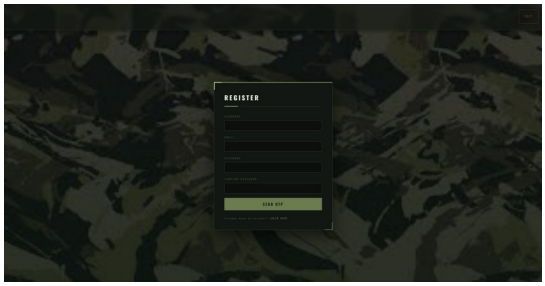
All authentication flows are handled via the auth blueprint (auth_routes.py) and rendered through the auth template base (authbase.html).

- Login (/login)
  A login form accepting username/email and password. On successful authentication, the user session is populated with user ID, username, and role. Failed login attempts are logged to the login_history table (with IP address, user agent, timestamp, and success flag). A login notification email is sent on each successful login.



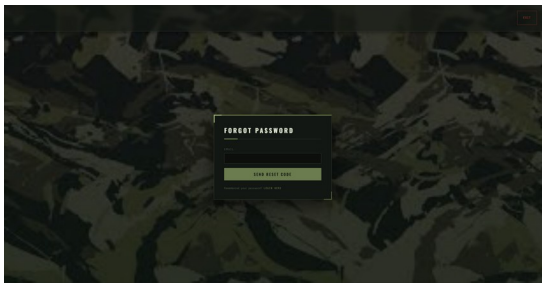
- Registration (/register)
  A registration form for new users. Collects username, email, and password. An OTP (one-time password) is sent to the provided email address via the email service for verification before the account is created. Passwords are hashed before storage using secure hashing (Werkzeug).

- Logout (/logout)
  Clears the user's session, effectively logging them out and redirecting them to the login page or homepage.



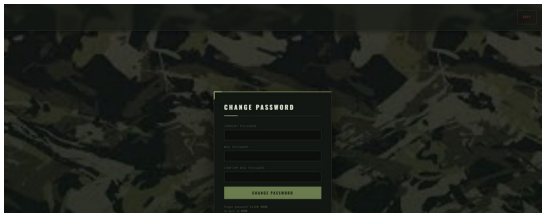
- Change Password (/change-password or /password)
  Allows a logged-in user to update their password. Requires the current password for verification before applying the new one.

- Forgot Password (/forgot-password)
  A password reset flow for users who have forgotten their credentials. The user provides their email address, an OTP is sent to that address, and upon successful OTP verification the user can set a new password.



- OTP Verification
  One-time passwords are generated (using the pyotp library) and emailed to the user for both registration verification and password reset flows.

- Login History Logging
  Every login attempt (successful or failed) is recorded to a login_history table, capturing the user ID, timestamp, IP address, user agent string,



and whether the attempt succeeded. This history is visible to the user in the Settings page under Privacy and Security.

Email Notifications

A dedicated email service module (email_service.py) handles all outbound transactional emails via SMTP (configured through environment variables). The service supports multi-port fallback (ports 443, 465, 587) to maximize deliverability in restricted network environments.

- Order Confirmation Email
  Sent to the customer automatically when a new order is successfully placed. Includes the order ID, a list of all purchased items with quantities and prices, and the order total.

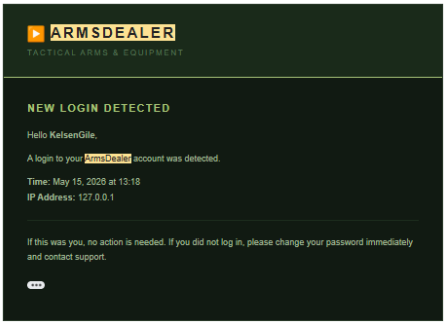
- Order Status Update Email
  Sent to the customer when an admin updates the status of their order (e.g., from pending to shipped, or shipped to delivered). Includes the new status and the order details.

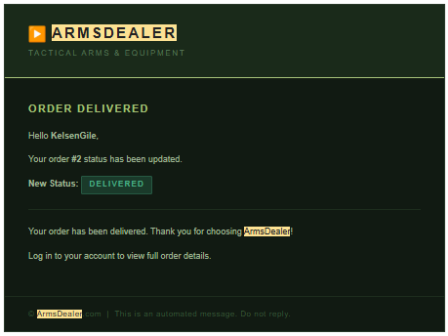
- Login Notification Email
  Sent to the user when a successful login is detected on their account. Provides the login timestamp and device/IP information as a security alert.

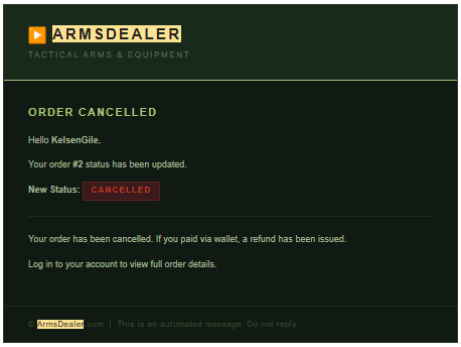
- OTP / Verification Email
  Sent during user registration and the forgot-password flow. Contains a time-limited one-time password code the user must enter to verify their email address or complete a password reset.

- Notification Preferences
  Each user has per-category notification preferences stored in the database. The email service checks these preferences before sending, respecting the user's choices about which notification types they wish to receive.

- Quiet Hours
  The email service supports a quiet hours configuration, suppressing non-critical notification emails during user-specified time windows.







Toast Notification System

A global client-side toast notification system (toast.js, toast.css partials/toast.html) that displays brief, non-blocking status messages to the user after actions such as adding items to the cart, updating settings, submitting forms, or encountering errors. Toasts appear temporarily and dismiss automatically.

Translations System

A client-side translation system powered by translations.js. All user-visible text strings in the templates are tagged with data-translate attributes referencing translation keys. When the user switches languages via the TopBar language selector, all tagged elements are updated dynamically without a page reload. Supported languages: English, Filipino, Japanese, Spanish, and Mandarin. Product names and descriptions also support server-side translations via a products_translations database table, which provides locale-specific names and descriptions pulled per request.

Currency Conversion System

All product and service prices are stored in PHP (Philippine Peso) as the base currency. A currencies table stores exchange rates for supported currencies (PHP, USD, EUR, GBP, SGD, JPY). On every page load, prices are multiplied by the active currency's rate and displayed with the appropriate symbol. The active currency is stored in a cookie and respected on every server render and dynamic price update.

Rating System

Users can rate products and services after their orders have been delivered. Star ratings (1–5) are submitted from the Orders History section and stored in the product_ratings table. Each user can only rate a given item once per item type. Aggregate ratings are reflected on product cards and detail pages via the rating field on the products and services tables.

System Requirements

Runtime Requirements

Python 3.10 or later (project developed and tested on Python 3.13)
pip (Python package installer)
A Unix-like operating system (Linux or macOS) for production deployment; Windows is supported for local development

Python Dependencies
The following packages are listed in requirements.txt and must be installed:

flask, version 3.0.0 or later — the core web framework
Werkzeug, version 3.0.0 or later — password hashing and security utilities, included as a Flask dependency
python-dotenv, version 1.0.0 or later — loads the .env configuration file
pyotp, version 2.9.0 or later — required only for the two-factor authentication feature; the application will still run without it but TOTP endpoints will return a 500 error if pyotp is not installed

Database
The application uses SQLite 3, which is included in the Python standard library. No separate database server is required. The database file is located at database/armsdealer.db within the project root.

Email (Optional)
A working SMTP server is required for transactional email functionality (registration OTP, login notifications, order confirmations, status updates, inquiry replies, and data export emails). The application is configured for Gmail SMTP by default. A Gmail App Password is required if using Gmail with 2-Step Verification enabled on the sending account.

If SMTP is not configured, the application falls back gracefully. OTP codes for registration and password reset are displayed directly in the browser as flash messages during development. All other email functions (notifications, confirmations, exports) will silently fail without affecting the core application workflow.

Browser
Any modern browser with JavaScript enabled. The application does not use any browser-specific APIs beyond standard cookies, fetch, and sessionStorage. No browser extensions are required.

Installation and Setup

Step 1: Extract or clone the project.
Place the ArmsDealer.com directory in your preferred working location.

Step 2: Create a virtual environment.
Navigate into the project root and run: python -m venv env

Step 3: Activate the virtual environment.
On Linux or macOS, run: source env/bin/activate
On Windows PowerShell, run: .\env\Scripts\Activate.ps1

Step 4: Install dependencies.
Run: pip install -r requirements.txt
To enable two-factor authentication, also run: pip install pyotp

Step 5: Configure environment variables.
Create a .env file in the project root (or edit the existing one) with the following values:

SECRET_KEY: a long random string for Flask session signing
SMTP_HOST: your SMTP server hostname
SMTP_PORT: the SMTP port (leave as 587 for Gmail STARTTLS)
SMTP_USER: your SMTP username (email address)
SMTP_PASS: your SMTP password or Gmail App Password
MAIL_FROM: the sender address for outgoing emails
SMTP_USE_TLS: set to true for STARTTLS

Step 6: Initialize the database.
If starting from a clean state, run: python init_db.py
This creates the database directory and runs schema.sql to create all tables.

Step 7: Seed the database.
Open the file database/armsdealer.db.sql in DB Browser for SQLite (or any SQLite client) and execute it against your armsdealer.db to populate the initial product, brand, category, currency, and language data.

Step 8: Run the application.
Run: python armsdealer.py
The application will start in debug mode and be accessible at http://127.0.0.1:5000

Development Accounts

The database is seeded with the following test accounts for development and testing purposes. These accounts should not exist in any production deployment.

Admin account:
Username: mrcrabs
Email: eugene.crabs@thekrustykrab.com
Password: MoneyM0ney!
Role: admin

Customer accounts:
Username: spongebob | Email: spongebob@bikini.bottom | Password: Krabby1234!
Username: patrickstar | Email: patrick.star@bikini.bottom | Password: IAmStupid!23
Username: squidward | Email: squidward.tentacles@bikini.bottom | Password: IHateEveryone!23
Username: sandycheeks | Email: sandy.cheeks@texas.com | Password: YeeHawTexas!23
Username: plankton | Email: sheldon.plankton@chumbucket.com | Password: SecretFormula!23
Username: ricksanchez | Email: rick.sanchez@c137.dim | Password: Wubbalubbadubdub!2
Username: mortysmith | Email: morty.smith@c137.dim | Password: Oh_Geez_Rick!23
Username: ben10 | Email: ben.tennyson@plumber.net | Password: Omnitrix!2310
Username: gwentennyson | Email: gwen.tennyson@plumber.net | Password: Anodite!Magic23
Username: grandpamax | Email: max.tennyson@plumber.net | Password: PlumberMax!2326

All of the above accounts and their associated data should be removed before any public or production deployment.

Known Limitations

SQLite concurrency. SQLite does not support concurrent writes. Under high simultaneous write load (many users placing orders or updating carts at the same time), write contention can cause errors. This is acceptable for small to medium deployments but would need to be addressed with a migration to PostgreSQL or MySQL for a high-traffic production environment.

No server-side session invalidation. Flask uses signed client-side cookies for sessions. When a session is "revoked" in the sessions table, the actual cookie on the user's browser is not invalidated. The revoke record serves as a transparency feature and a foundation for enforcement, but a logged-in session cookie would still be valid until it expires. True remote logout requires a server-side session store such as Flask-Session with Redis.

Account deactivation is not enforced at login. The is_active flag is set to 0 by the deactivate endpoint, but the login route does not currently check this flag. A deactivated user can still log in. This check would need to be added to the login route's credential verification block to fully enforce deactivation.

SMTP credentials in .env. The .env file contains the SMTP password in plaintext. The .env file is listed in .gitignore and should never be committed to version control. In production, credentials should be injected via the hosting environment's secret management system rather than a file on disk.
models.py is partially implemented. The models.py file defines static helper methods for User, Category, and Product but is not imported or used by any route. All database queries are written inline in the route files. The models file appears to be an early-stage refactoring effort that was not completed.

No input sanitization beyond basic validation. The application validates required fields and data types but does not apply HTML sanitization on user-submitted text fields such as bio, delivery address, or inquiry messages. These fields are passed to Jinja2 templates which auto-escape output by default, so XSS via template rendering is prevented. However, for the admin dashboard where raw data is displayed, additional sanitization should be applied before any production deployment.

No rate limiting on login. The login route does not implement brute-force protection such as account lockout after N failed attempts, CAPTCHA, or IP-based rate limiting. Failed login attempts are logged, but no automated blocking occurs.

No HTTPS enforcement. The application does not enforce HTTPS at the application layer. In production, HTTPS must be terminated at the reverse proxy level (nginx, Caddy, or a cloud load balancer), and the Flask app should be configured with SESSION_COOKIE_SECURE = True and PERMANENT_SESSION_LIFETIME to enforce secure cookie behavior.

Recommendations

Immediate Before Any Production Deployment

Replace all development test accounts (the SpongeBob and Rick and Morty accounts) with real administrative users and remove the plaintext credentials from notes.txt.

Set a strong, randomly generated SECRET_KEY in the .env file. The development placeholder must never be used in production as it compromises all Flask session security.

Set SESSION_COOKIE_SECURE = True, SESSION_COOKIE_HTTPONLY = True, and SESSION_COOKIE_SAMESITE = 'Lax' in the Flask configuration to harden cookie security.

Serve the application behind a production-grade WSGI server such as Gunicorn with at least 2 workers, behind a reverse proxy such as nginx configured for HTTPS termination with a valid TLS certificate.
Move SMTP credentials out of the .env file and into the server environment's secret management if using a cloud hosting provider.

Remove the debug=True flag from the armsdealer.py run call. Debug mode exposes an interactive debugger over HTTP and must never be enabled in production.

Short-Term Improvements

Add a login attempt lockout mechanism. After 5 consecutive failed login attempts from the same IP or for the same account, temporarily block further attempts and notify the user.

Add the is_active check to the login route so that deactivated accounts cannot log in.
Implement server-side session storage (Flask-Session with Redis or a database backend) to enable true remote session invalidation when a user revokes a session.

Complete the models.py refactoring and route all database queries through the model layer rather than writing raw SQL inline in route handlers. This will make the codebase more maintainable and testable.

Add input sanitization using a library such as bleach for user-submitted text that may be rendered in the admin dashboard or in email content.

Add pagination to the admin dashboard's orders table, product inventory table, and users table. As the dataset grows, loading all records at once will become a performance bottleneck.

Medium-Term Improvements

Consider migrating from SQLite to PostgreSQL for any multi-user production deployment. PostgreSQL supports concurrent writes, row-level locking, and connection pooling, all of which are necessary for reliable performance under real traffic.

Implement a proper task queue (Celery with Redis, or a lightweight alternative such as rq) for sending transactional emails asynchronously rather than blocking the HTTP response cycle.

Add automated testing. The project currently has no test suite. Unit tests for the authentication flow, cart operations, and order management would significantly reduce the risk of regressions during future development.

Add request logging and error tracking. Integrating a service like Sentry for exception tracking and structuring server logs with proper log levels will make production issues much easier to diagnose.

Security Considerations

Password storage. All passwords are stored as Werkzeug PBKDF2-SHA256 hashes. Plain-text passwords are never stored or logged.

Session security. Flask sessions are signed with SECRET_KEY using HMAC. Session data cannot be tampered with by the client, but the session cookie itself should be protected with Secure and HttpOnly flags in production.

SQL injection. All database queries use parameterized statements with ? placeholders. No string interpolation or f-string formatting is used for user-supplied values in SQL queries except in one location in the search API where is_logged_in controls a clause toggle. This usage is safe because it is determined by server-side session state rather than user input.

File uploads. Profile image, product image, service image, and brand logo uploads are validated for file extension against a whitelist (png, jpg, jpeg, gif, webp) using werkzeug.utils.secure_filename. Uploaded filenames are prefixed with a type and ID prefix to prevent path traversal. Upload directories are created with os.makedirs if they do not exist.

CSRF protection. The application does not currently implement CSRF token protection. For state-changing form submissions and API endpoints, CSRF tokens should be added using Flask-WTF or a custom implementation, particularly for the checkout and account deletion endpoints.

OTP expiry. Registration and password reset OTP codes expire after 10 minutes. The expiry is enforced server-side by comparing the current UTC time against the stored otp_sent_at timestamp.

Authorization checks. All admin API endpoints verify both authentication (user_id in session) and role (role == 'admin') before performing any action. User-specific endpoints verify that the requesting user owns the resource before allowing modification.

Email credentials. The SMTP_PASS credential in the committed .env file is a Google App Password associated with the developer's personal email. This credential should be rotated immediately if the repository has been shared or made public, and should be replaced with a dedicated platform email account and credential before any production deployment.

Performance Considerations

The database schema includes indexes on all foreign key columns and on the most frequently queried fields including orders.status, orders.user_id, products.category_id, products.brand_id, products.subcategory_id, services.category_id, and order_items.order_id. These indexes ensure that common queries (filtering by category, fetching user orders, joining order items) remain fast as the dataset grows.

The homepage product and service queries use hardcoded ID lists rather than dynamic queries, ensuring consistent fast response times for the most frequently accessed page.

The admin dashboard loads all products, services, users, and orders in a single page request. This approach is acceptable for small datasets but will become slow as inventory and user counts grow into the thousands. Pagination and lazy loading should be implemented in the dashboard before the dataset reaches that scale.

The email sending function runs synchronously within the request cycle. For pages that send email (login, checkout, order status update, inquiry reply), the response is delayed by the SMTP connection time, which can range from under a second to several seconds depending on the SMTP server. Moving email sending to a background task queue would eliminate this latency.
Static files (CSS, JavaScript, images) are served by Flask's built-in static file handler, which is not optimized for production use. In production, static files should be served directly by nginx or through a CDN to reduce load on the application server.

Scalability Considerations

The current architecture is a single-process Flask application with a single SQLite database file. This is appropriate for development, demonstration, and low-traffic deployments.
For scaling beyond a single server, the following changes would be required:

The database must be migrated from SQLite to a client-server database such as PostgreSQL. SQLite does not support multiple simultaneous connections from different processes or machines.

The Flask session must be moved to a shared session store (Redis or a database table) so that sessions are consistent across multiple application server instances.

The uploaded file storage (profile images, product images, brand logos) must be moved from the local filesystem to a shared object storage service such as Amazon S3 or a compatible alternative, so that all application instances can access the same files.

The email sending must be moved to a background task queue that any instance can push jobs to, with a single worker pool consuming the queue.

A load balancer would distribute incoming HTTP traffic across multiple application server instances, each running Gunicorn with multiple workers.

Future Enhancements

The following features are planned or proposed for future development based on the project notes and the current state of the codebase.

Promotions System

A complete promotions system is planned. This would include bundles (grouped products sold together at a discounted price), vouchers and discount codes, seasonal sales tied to calendar events, exclusive limited-edition drops with quantity limits, event-based sales, and product rankings and best-seller lists. This feature would require new database tables for promotions, voucher codes, and bundle definitions, as well as new checkout logic to apply discounts before order creation.

Services Browsing Page

The services page currently renders a static template. A full browsing experience similar to the products page is planned, with filtering by category (authorized vs. restricted), service provider, and price range. Individual service detail pages need to be completed and linked from the services listing.

Tools Section

A tools section is planned for the navigation bar. The intended scope of this section is not fully documented, but it could include range calculators, ballistics tools, gear comparison utilities, or other interactive resources relevant to the target user base.

News and Updates Section

A news and updates section on the homepage and as a dedicated page is planned but not yet implemented. This would likely require a new articles or posts database table with title, body, publish date, author, and category fields, along with an admin interface for creating and managing posts.

Best Sellers and Trusted Brands

The homepage has placeholder sections for best sellers (products ranked by sales_count) and a trusted brands carousel. The underlying sales_count data already exists in the database; the work needed is primarily front-end implementation of these homepage sections.

Product Ranking, Sorting, and Filtering Controls

The products browsing page lacks user-facing controls for sorting by price, rating, or sales count, and for filtering by subcategory, price range, or stock availability. These controls exist conceptually in the API (the search API supports ordering by sales_count) but need to be exposed in the products page UI.

Promotional Vouchers and Wallet Top-Up

The wallet system exists and is used for order payment, but there is no mechanism for users to add funds to their wallet within the application. A top-up flow (potentially integrated with a payment gateway) and a voucher redemption system would make the wallet more useful to end users.

Payment Gateway Integration

The current payment system only supports cash on delivery and internal wallet balance. Integration with a real payment gateway (such as PayMongo for PHP markets, Stripe, or PayPal) would be required for a fully functional commercial deployment.

Mobile-Responsive Layout Refinements

While the application uses a custom CSS framework and is not inherently broken on mobile, specific pages (particularly the admin dashboard with its large data tables) are not optimized for small screen sizes. A dedicated mobile layout for the dashboard and checkout flow would improve usability on phones and tablets.

Inventory Alerts and Low Stock Notifications

The admin dashboard already shows a low stock panel for products with 5 or fewer units. An automated email alert to admins when a product falls below a configurable threshold would help prevent stockouts without requiring manual dashboard monitoring.

Order Tracking Page

A public-facing order tracking page where customers can check their order status by entering an order ID (without logging in) would improve the customer experience, particularly for users who placed orders as guests (if guest checkout is ever added).

Audit Log

A complete audit log recording all admin actions (product additions, price changes, order status updates, user role changes, and account deletions) would improve accountability and make it possible to review the history of any record change.

API Versioning and Documentation

The current API endpoints have no versioning. Adding a version prefix (such as /api/v1/) would allow future breaking changes to be made without disrupting existing integrations. Formal API documentation using OpenAPI/Swagger would make the API easier to consume by any future frontend applications or mobile clients.

Brands Catalogue Reference

The platform's database is seeded with a comprehensive set of real-world arms, defense, and security brands organized across multiple industry segments.

Firearms manufacturers: Glock (Austria), Colt (USA), Heckler & Koch (Germany), Sig Sauer (global), FN Herstal (Belgium), Beretta (Italy), Walther (Germany), Mauser (Germany), Kalashnikov Concern (Russia), Remington Arms (USA), Smith & Wesson (USA), Springfield Armory (USA), Steyr Mannlicher (Austria), IMI Systems (Israel), Accuracy International (UK), Winchester (USA).

Bladed weapons: Benchmade, KA-BAR, Cold Steel, Leatherman.
Less-lethal and law enforcement: Axon (Tasers), Combined Systems, Defense Technology, ASP (batons), Safariland, Peerless Handcuff.

Ammunition: Hornady, Federal Premium, Winchester.
Optics and sights: Trijicon, EOTech, Vortex Optics, FLIR Systems.

Tactical gear and apparel: 5.11 Tactical, Crye Precision, Magpul, SureFire.
Protection and armor: 3M, Armor Express.

Weapon attachments and maintenance: Magpul, Otis Technology, Hoppe's, ZEV Technologies, Agency Arms.
Storage and transport: Liberty Safe, Pelican, Iron Mountain.

Communication and electronics: Motorola Solutions, Garmin, Hikvision, Garrett Metal Detectors.

Defense systems and aerospace: Raytheon, Lockheed Martin, Northrop Grumman, General Dynamics, L3 Technologies.

Chemical and biological defense: Thermo Fisher Scientific, Mirion Technologies, Veolia.

Vehicles: Oshkosh Defense, Polaris Defense.

Cybersecurity: CrowdStrike, Palo Alto Networks.

Power and survival: Duracell, Goal Zero, LifeStraw.

Training and simulation: Bluegun, Action Target.

Security services: Academi, G4S, Allied Universal, Control Risks, Brink's, GardaWorld.

Research and advisory: MITRE, UL Solutions, SGS, KBR.

Anonymous: A special brand for sellers whose identity is not disclosed or verified.

Categories and Products Reference

Products are organized into two top-level types: weapons and equipment. Services form a separate top-level type.

Weapons Subcategories

Firearms covers handguns, rifles, shotguns, submachine guns, machine guns, sniper rifles, carbines, and personal defense weapons. Blades covers knives, swords, daggers, machetes, bayonets, axes, hatchets, combat spears, polearms, and throwing knives. Blunts covers batons, clubs, maces, hammers, flails, nunchaku, staves, saps, and blackjacks. Projectiles covers bows, crossbows, slingshots, spear launchers, blowguns, javelin systems, and pneumatic launchers. Explosive covers grenades, bombs, mines, demolition charges, rocket launchers, mortars, artillery shells, breaching charges, and flash-bangs. Electronic covers tasers, stun guns, EMP devices, jamming devices, directed energy weapons, sonic devices, and laser dazzlers. Chemical covers tear gas, smoke agents, incendiary compounds, pepper sprays, aerosol dispersants, and riot control agents. Biological covers delivery systems, containment units, detection equipment, and neutralization agents. Vehicle covers armed vehicles, drones, naval weapon systems, aerial weapon systems, and anti-vehicle weapons. Cyber covers malware tools, exploit kits, hacking devices, intrusion systems, data exfiltration tools, and ransomware platforms. Security covers access control weapons, crowd control devices, non-lethal weapons, deterrent systems, and restraint devices.

Equipment Subcategories

Ammunition covers bullets, shells, cartridges, energy cells, primers, propellants, specialty rounds, and caseless ammunition. Protective covers helmets, body armor, shields, protective suits, ballistic glasses, hearing protection, gas masks, and blast-resistant gear. Tactical covers load-bearing gear, holsters, utility belts, field kits, chest rigs, plate carriers, drop leg platforms, and modular pouches. Optics covers scopes, red dot sights, night vision devices, thermal imagers, rangefinders, binoculars, and spotting scopes. Attachments covers suppressors, grips, laser sights, foregrips, bipods, muzzle brakes, flashlights, and bayonet mounts. Maintenance covers cleaning kits, repair tools, lubricants, bore snakes, armorer tools, parts kits, and solvent traps. Storage covers safes, cases, lockers, ammunition cans, weapon racks, concealment furniture, and transport containers. Communication covers radios, signal devices, encryption units, earpieces, tactical headsets, satellite communicators, and covert devices. Survival covers first aid kits, rations, water purification, navigation tools, emergency shelters, fire starters, multi-tools, and tourniquets. Training covers simulators, dummy equipment, targets, training manuals, blue guns, force-on-force gear, and recoil trainers.

Services Categories

Manufacturing covers mass production, custom fabrication, prototyping, OEM production, contract manufacturing, and component supply. Customization covers weapon modding, gear personalization, engraving, cerakoting, stock fitting, and trigger jobs. Maintenance covers repair services, inspection, refurbishment, preventive maintenance, depot-level overhaul, and parts replacement. Transport covers secure logistics, international shipping, escort services, armored courier, cold chain transport, and cross-border brokerage. Storage covers secure warehousing, vault services, inventory management, climate-controlled storage, evidence storage, and bonded warehousing. Training covers tactical training, safety training, technical training, marksmanship courses, force-on-force exercises, and simulator programs. Protection covers personal security, facility security, escort protection, executive protection, maritime security, and event security. Consulting covers risk assessment, security planning, compliance advisory, threat analysis, regulatory consulting, and vulnerability audits. Research covers product development, field research, technology innovation, ballistics research, materials science, and weapons systems R&D. Testing covers performance testing, safety testing, certification, ballistic testing, environmental testing, and proof testing. Disposal covers decommissioning, hazardous disposal, recycling, demilitarization, ammunition destruction, and EOD services. Surveillance covers monitoring services, reconnaissance, intelligence gathering, aerial surveillance, counter-surveillance, and SIGINT services. Contracting covers private contracts, government contracts, defense contracts, subcontracting, joint venture agreements, and grant procurement.

Database Maintenance

Resetting Cart and Order Data

To clear all cart, order, rating, and inquiry data without dropping the full database (useful during development and testing), execute the following SQL against the database:

```
PRAGMA foreign_keys = OFF;
DELETE FROM cart_items;
DELETE FROM order_items;
DELETE FROM orders;
DELETE FROM product_ratings;
DELETE FROM inquiries;
DELETE FROM sqlite_sequence WHERE name='cart_items';
DELETE FROM sqlite_sequence WHERE name='order_items';
DELETE FROM sqlite_sequence WHERE name='orders';
DELETE FROM sqlite_sequence WHERE name='product_ratings';
DELETE FROM sqlite_sequence WHERE name='inquiries';
PRAGMA foreign_keys = ON;
```

This preserves all user accounts, products, services, brands, and categories while returning the transactional data to a clean state.

Rebuilding the Database from Schema

To rebuild the database from scratch, run init_db.py. This will execute schema.sql and create all tables, indexes, and triggers. After running init_db.py, execute the seed data from armsdealer.db.sql to repopulate the catalogue and reference data.

Auto-Migrating Columns

Several newer columns were added to existing tables after the initial schema was written (admin_reply, replied_at, order_number on inquiries; notification_prefs on users; login_history and user_sessions and user_2fa as entirely new tables). The application handles these additions automatically using CREATE TABLE IF NOT EXISTS and ALTER TABLE ... ADD COLUMN IF NOT EXISTS patterns within the route code. No manual migration commands need to be run when updating from an older version of the database.

Contribution Notes

The project is organized as a standard Flask application with blueprints. New routes should be added to the appropriate existing blueprint (auth, main, cart, or api) or a new blueprint if the feature domain is sufficiently distinct. New blueprints must be registered in armsdealer.py.

Database queries are currently written inline in route handlers. The models.py file was started as a centralized query layer but is not currently in use. New feature work should prefer adding methods to models.py and importing them in route handlers, as this improves testability and reduces code duplication.

All state-changing API endpoints should verify authentication and, where applicable, authorization against the target resource before performing any database write. The existing _admin_required() helper function in api_routes.py provides a reusable pattern for admin-only endpoints.

Email sending should always be wrapped in a try-except block and should never be allowed to raise an exception that would interrupt the request cycle. All email calls in the existing codebase follow this pattern with pass in the except handler and a comment noting the intent.

The appearance system injects CSS directly into every page. When adding new CSS variables or theming hooks, the inject_appearance context processor in armsdealer.py must be updated alongside the template and stylesheet changes.

Definition of Terms

Access Control — A security mechanism within the platform that regulates which users can view or interact with specific content. In ArmsDealer.com, access control determines whether a visitor can see restricted product listings based on their authentication status.

Admin — A user role with elevated privileges that grants full access to the admin dashboard, including the ability to manage products, services, brands, users, orders, and support inquiries. Assigned via the role field in the users table.

Admin Dashboard — A protected web interface accessible only to users with the admin role. It provides a centralized panel for managing all operational aspects of the platform including inventory, orders, customers, analytics, and inquiries.

Appearance System — A server-side theming system that reads user preferences from a browser cookie and injects customized CSS into every page render. Controls color mode, accent color, background image, font size, scanlines, compact mode, and animations.

Authorization Flag (is_authorized) — A binary field on product, service, and brand records that controls public visibility. Items with is_authorized set to 1 are visible to all visitors. Items set to 0 are visible only to logged-in users.

Blueprint — A Flask architectural concept used to organize routes into modular, reusable components. ArmsDealer.com uses four blueprints: auth, main, cart, and api, each handling a distinct domain of the application.

Brand — A manufacturer or supplier entity associated with one or more products. Each brand has a name, slug, logo image, authorization flag, country of origin, and description. Brands are managed through the admin dashboard.

Cart — A persistent database record tied to a user account that holds items the user intends to purchase. Cart items include a type (product or service), item ID, and quantity. The cart survives session expiry and browser closure.

Category — The top-level classification of a product or service in the catalogue hierarchy. Categories are typed as either product or service and contain multiple subcategories. Examples include Firearms, Blades, Ammunition, and Training.

Checkout — The process through which a user reviews their cart, selects a payment method, and places an order. Checkout converts the cart into a formal order record and clears the cart upon completion.

Cookie — A small data file stored in the user's browser used by the platform to persist preferences across sessions. ArmsDealer.com uses cookies to store the selected currency, selected language, and appearance settings.

Currency Conversion — The process of multiplying a product's base PHP price by the exchange rate of the user's selected currency for display purposes. Conversion is applied at render time and never stored in the database.

Customer — The default user role assigned to all newly registered accounts. Customers can browse products, manage a cart, place orders, and access their account settings. They do not have access to the admin dashboard.

Delivery Address — A user profile field storing the customer's preferred shipping address. Populated in account settings and referenced during order processing.

Discount — A percentage reduction applied to a product or service's base price. Stored as a real number in the discount column and used to calculate the displayed sale price.

Flask — The Python micro web framework used as the foundation of ArmsDealer.com. Flask handles HTTP routing, session management, template rendering via Jinja2, and request/response processing.

Flash Message — A one-time notification message displayed to the user on the next page load after a form submission or redirect. Used throughout the platform to communicate success, warning, and error states.

Forgot Password — A password recovery flow that verifies the user's identity by sending a time-limited OTP to their registered email address before allowing a password reset.

Guest — An unauthenticated visitor who has not logged in. Guests can browse the public catalogue and submit inquiries but cannot add items to a cart, place orders, or view restricted product listings.

Inquiry — A support message submitted by a user or guest through the contacts page. Inquiries are stored in the inquiries table with a subject, message, and optional order reference. Admins can reply to inquiries by email from the dashboard.

is_active — A flag on the users table that indicates whether an account is active. Set to 0 when a user deactivates their account. Intended to prevent login for deactivated accounts.

Jinja2 — The templating engine used by Flask to render dynamic HTML pages. Jinja2 processes template files containing variables, loops, and conditionals, replacing them with server-generated content before sending the response to the browser.

JWT (JSON Web Token) — Not currently used in ArmsDealer.com. Authentication is handled through server-side Flask sessions rather than stateless tokens.

Login History — A record of every login attempt made by a user, stored in the login_history table. Each entry captures the timestamp, IP address, user agent string, and whether the attempt was successful.

Multi-Currency Support — A feature allowing users to view all prices in their preferred currency. Supported currencies include PHP, USD, EUR, GBP, SGD, JPY, and CNY. The selected currency is persisted in a browser cookie.

Multi-Language Support — A feature enabling product names, descriptions, brand names, category names, and UI strings to be displayed in the user's selected language. Translations are stored in dedicated translation tables in the database.

Notification Preferences — User-configurable settings stored as JSON in the users table that control which categories of email the user receives, including order updates, security alerts, and promotional messages.

Order — A formal purchase record created when a user completes checkout. Orders have a status, a total in PHP, a reference to the user, and a list of associated order items. Orders are tracked through five stages: order placed, packing, shipping, delivered, and cancelled.

Order Item — An individual line within an order representing a single product or service, its quantity, and the unit price at the time of purchase. Stored in the order_items table linked to a parent order.

OTP (One-Time Password) — A six-digit numeric code generated by the server and sent to the user's email address. Used to verify identity during account registration and password reset. Codes expire after 10 minutes.

Payment Method — The method selected at checkout to pay for an order. ArmsDealer.com currently supports cash on delivery and wallet payment. The selected method is recorded in the order's notes field.

PHP (Philippine Peso) — The base currency of the platform. All prices in the database are stored in PHP. All other currency values are computed by multiplying the PHP price by the relevant exchange rate at display time.

Product — A physical item listed in the catalogue for sale. Each product belongs to a category, subcategory, and optionally a brand. Products have price, discount, stock, rating, sales count, authorization flag, tags, and up to five associated images.

Product Images — Additional image files associated with a product, stored in the product_images table. A product can have up to five images, with the primary image also stored directly in the products table's image_file field.

Product Rating — A star-based score (1 through 5) submitted by a customer who has received a delivered order containing that product or service. Ratings are stored in the product_ratings table and the product's average rating is recomputed and written back to the products table after each submission.

Profile Image — A user-uploaded photograph or avatar stored in the userimages directory and referenced by filename in the users table. Displayed in the account panel and account page.

Restricted Item — A product, service, or brand with is_authorized set to 0. These items are hidden from unauthenticated visitors and from the guest-accessible search results. Only logged-in users can view restricted listings.

Role — A field on the users table that defines a user's permission level within the platform. Valid values are customer and admin.

Sales Count — A field on the products and services tables that tracks how many units have been sold across all delivered orders. Incremented automatically when an order's status transitions to delivered.

Service — A non-physical offering listed in the catalogue, such as training, maintenance, transport, or consulting. Services follow the same structure as products but do not have a stock field.

Session — A server-side data store tied to a signed browser cookie that persists user state across requests. Flask sessions in ArmsDealer.com store the user's ID, username, email, role, cart count, wallet balance, profile data, and social links.

Session Management — The system for tracking and managing active browser sessions per user. Implemented through the user_sessions table, which records device type, IP address, user agent, and last-seen timestamp for each session. Users can revoke individual or all non-current sessions from their security settings.

Slug — A URL-safe identifier derived from a record's name, used in place of numeric IDs in public-facing URLs. For example, a product named "Glock 17" might have the slug glock-17, used in the URL /product/glock-17.

SMTP (Simple Mail Transfer Protocol) — The protocol used by the platform to send transactional emails. Configured via the .env file and used for registration OTPs, login notifications, order confirmations, status updates, inquiry replies, and data exports.

SQLite — The embedded relational database engine used by ArmsDealer.com. The entire database is stored in a single file (armsdealer.db). SQLite is suitable for development and low-to-medium traffic deployments but does not support concurrent writes from multiple processes.

Stock — An integer field on product records representing the number of units currently available for purchase. Stock is automatically reduced when an order containing that product is moved to the shipping status.

Subcategory — A second-level classification within a parent category. For example, the Weapons category contains subcategories such as Firearms, Blades, and Explosive. Subcategories can be filtered on the products browsing page.

TOTP (Time-Based One-Time Password) — A standard algorithm (RFC 6238) that generates short-lived numeric codes from a shared secret and the current time. Used by ArmsDealer.com's two-factor authentication feature, compatible with authenticator applications such as Google Authenticator and Authy.

Two-Factor Authentication (2FA) — An optional additional security layer requiring users to provide a TOTP code from their authenticator application in addition to their password when logging in. Managed via the user_2fa table and the pyotp library.

UI Strings — Internationalized text labels used throughout the interface, stored in the ui_strings table keyed by language code and string key. These allow the platform's interface text to be displayed in the user's selected language.

Wallet — A stored PHP balance associated with each user account. Can be used as a payment method at checkout. The balance is deducted immediately upon order placement and refunded automatically if an eligible order is cancelled.

Wallet Balance — The current PHP amount stored in a user's internal wallet, held in the wallet_balance field of the users table. Displayed in the selected currency on the account and settings pages.

Werkzeug — A Python library used by Flask that provides password hashing utilities, secure filename handling for uploads, and WSGI utilities. ArmsDealer.com uses Werkzeug's generate_password_hash and check_password_hash functions for all credential storage and verification.

WSGI (Web Server Gateway Interface) — A Python standard defining how web servers communicate with Python web applications. Flask is a WSGI application. In production, it should be served by a WSGI server such as Gunicorn rather than Flask's built-in development server.

Bibliography

Grinberg, Miguel. Flask Web Development: Developing Web Applications with Python. 2nd ed. O'Reilly Media, 2018.

Pallets Projects. Flask Documentation, Version 3.1. https://flask.palletsprojects.com, 2024.

Pallets Projects. Werkzeug Documentation. https://werkzeug.palletsprojects.com, 2024.

Python Software Foundation. sqlite3 — DB-API 2.0 Interface for SQLite Databases. Python 3.13 Standard Library

Documentation. https://docs.python.org/3/library/sqlite3.html, 2024.

Python Software Foundation. smtplib — SMTP Protocol Client. Python 3.13 Standard Library Documentation. https://docs.python.org/3/library/smtplib.html, 2024.

Johansson, Per, and Adam Lindqvist. python-dotenv Documentation. https://saurabh-kumar.com/python-dotenv, 2024.

M2Crypto Contributors. pyotp — Python One-Time Password Library. https://pyauth.github.io/pyotp, 2024.

SQLite Consortium. SQLite Documentation. https://www.sqlite.org/docs.html, 2024.

OWASP Foundation. OWASP Top Ten Project. https://owasp.org/www-project-top-ten, 2021.

OWASP Foundation. Authentication Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html, 2024.

OWASP Foundation. Session Management Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html, 2024.

RFC 6238 — TOTP: Time-Based One-Time Password Algorithm. Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc6238, 2011.

RFC 5321 — Simple Mail Transfer Protocol. Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc5321, 2008.

Google. Gmail SMTP Settings and App Passwords. https://support.google.com/mail/answer/7126229, 2024.